

TREASURY SHIELD

a one-week track 1
mvp for crypto
treasury protection

the pitch in one sentence

**web3 teams hold volatile assets, but their expenses are fixed.
Treasury Shield makes the downside visible and helps them evaluate an affordable
Thetanuts options hedge before they put their runway at risk.**

prepared for team discussion

This brief translates the MUBA Hacks 2026 materials and Thetanuts builder guidance into a focused product proposal. It is a discussion document, not the final Devfolio submission.

the decision we are proposing

Build Track 1 first: a frontend-only, simulation-first treasury risk product using live Thetanuts market data. Do not build an autonomous AI trader in the one-week sprint.

1. the story: alice and bob

Alice runs a small web3 studio. The studio holds 10 ETH in its treasury and must pay contractors next week. Bob manages a DAO treasury with a similar problem: the DAO has crypto assets, but its grant and contributor expenses are priced in dollars.

Today, Alice and Bob see a balance in their wallet, but they do not see their real runway. If ETH drops sharply, they may have to sell more ETH, delay payments, or cut planned work. They hear the word “options,” but the existing terminology, strike tables, and payoff charts are difficult to use under pressure.

They do not want to become professional options traders. They want to answer one practical question:

“will our treasury still cover our next expenses if crypto falls, and what would protection cost?”

the problem statement

small web3 teams often hold treasury funds in volatile crypto assets while their operating costs are fixed. Existing options tools are designed around trading vocabulary instead of business outcomes, leaving treasury managers without a simple way to measure runway risk, compare protection costs, and understand the result before acting.

the solution

Treasury Shield translates a treasury manager’s business situation into a clear risk view. It combines treasury assumptions, live Thetanuts order data, price-drop scenarios, and a simple hedge recommendation. The first version ends with a simulated trade so the team can demonstrate the complete decision workflow without using real funds.

2. what we are building

Treasury Shield is a single-user web app with five focused screens:

screen	what the judge sees
1. setup	asset, treasury amount, upcoming expenses, protection period, and maximum premium
2. exposure	current treasury value, expense coverage, runway gap, and risk status
3. scenarios	unprotected treasury value under 5%, 10%, and 20% price drops
4. recommendation	one Thetanuts-based strategy with price, expiry, cost, and worst-case explanation
5. simulation	a simulated fill and before/after result using current market data

mvp acceptance test

- a judge can enter a treasury and expense amount in under one minute.
- the app loads current Thetanuts order or market data instead of hard-coded fake prices.
- the app shows how the same treasury behaves with and without a hedge.
- the recommendation explains the outcome in normal business language.
- the simulated flow ends with a clear summary of premium, expiry, protection, and remaining risk.

what is deliberately out of scope

No accounts, database, backend, AI agent, lending, vaults, automatic rebalancing, multi-chain support, complex Greeks, or real-money execution in the first demo.

3. why this qualifies for Thetanuts track 1

The track is for a real working product built on the Thetanuts SDK. Thetanuts must be load-bearing: if its orders, strikes, expiries, premiums, and payout logic were removed, our recommendation would not work.

track 1 expectation	treasury shield response
working product	deployed web app with a complete setup-to-simulation journey
meaningful sdk use	live Thetanuts market data powers available strategy and cost calculations
real user problem	protecting payroll and runway for small web3 teams
clear product fit	business outcome first, options complexity hidden behind explanation
presentation	one story, one user journey, one visible before/after result

track 1 versus track 2

Track 2 is a different promise: an AI agent must execute at least one real on-chain options trade on Base mainnet. That creates extra wallet, key-management, AI, confirmation, and real-funds risk. We are not choosing it simply because it has a larger-sounding technical story; our investment is in making the Track 1 product credible, understandable, and complete.

4. one-week technical stack

layer	choice	reason
frontend	Next.js + TypeScript	fast product user interface (UI) and simple Vercel deployment
components	HeroUI + Tailwind	accessible inputs, cards, tabs, alerts, and responsive user experience (UX)
charts	Recharts	scenario comparison and treasury runway visuals
protocol	Thetanuts client sdk	read live orders and market data from the protocol
chain reads	ethers or viem	read Base wallet balances and format transaction data
persistence	localStorage	save simulations without accounts or a database
hosting	Vercel + GitHub	one-click public demo and transparent commit history

data flow

```
browser UI -> Thetanuts client sdk -> live Base market data
              -> treasury calculations -> recommendation
              -> simulated hedge -> localStorage
```

sdk integration shape

```
const client = new ThetanutsClient({
  chainId: 8453,
  provider: baseProvider,
})

const orders = await client.api.fetchOrders()
const market = await client.api.getMarketData()

const candidates = filterOrders(orders, {
  asset: selectedAsset,
  expiry: protectionDays,
})

const recommendation = rankByCostAndProtection(
  candidates, treasuryInputs
)
```

important note

The pseudocode is a product-shape example, not a promise that every method name is exact. The team should follow the current Thetanuts repository and docs as the source of truth, as the builder document instructs.

5. security and trust

- simulation-first: no real-money transaction is needed for our demo flow.
- non-custodial: never ask for a seed phrase or private key.
- chain lock: support Base only and reject the wrong network.
- contract validation: keep verified token and protocol addresses in one constants file.
- clear disclosure: show premium, expiry, maximum loss, and remaining risk before simulation.
- input limits: reject impossible amounts, negative expenses, and unreasonable protection periods.
- slow polling: load market data on demand instead of creating a noisy polling loop.
- honest labels: separate live market data from estimated simulation output and include a not-financial-advice notice.

6. equal team split: 25% each

Each person owns one complete workstream. The percentage means equal ownership of the build, not that one person only performs one small task. Everyone joins the final integration, testing, and pitch rehearsal.

owner	25% workstream	definition of done
person 1	product flow and interface	setup, dashboard, scenario, recommendation, and simulation screens are usable and responsive
person 2	Thetanuts integration	live orders and market data are loaded, filtered, normalized, and shown with clear error states
person 3	treasury and hedge logic	runway, downside scenarios, hedge estimates, and recommendation ranking are tested
person 4	quality, documentation, and pitch	readme, test checklist, demo script, judging map, and final presentation are ready

shared handoff rules

- person 2 publishes the normalized market-data shape before person 1 and person 3 connect their screens.
- person 3 publishes calculation examples using fixed sample inputs before the live data is wired in.
- person 1 keeps the interface connected to shared types instead of hard-coded component-specific shapes.
- person 4 tests the full journey every day and can cut features that threaten the deadline.

7. four-to-five-day maximum sprint

day	shared milestone	parallel ownership
1	foundation and first screen	1: app shell and setup; 2: SDK spike; 3: formulas; 4: test cases and demo story
2	working data and calculations	1: dashboard; 2: live order adapter; 3: scenario engine; 4: integration checklist

day	shared milestone	parallel ownership
3	complete user journey	1: recommendation and simulation UI; 2: data states; 3: hedge comparison; 4: end-to-end test and readme
4	polished demo candidate	all four integrate; 1: responsive polish; 2: reliability; 3: numerical review; 4: pitch and video
5	freeze, deploy, and submit	critical fixes only; final demo rehearsal, public repo check, video, and submission package

stop rule

If the core flow works by day 3, do not expand the product. Use days 4 and 5 for reliability, clarity, visual polish, demo practice, and submission quality. Any feature that requires a backend, database, AI agent, private key, or real-money trade is outside this sprint.

8. what the wider team needs to know

MUBA Hacks 2026 is a build-and-demo competition. Teams must be 2–4 people, the project must be created during the official hacking period, and the submission needs a public repository, readme, demo video, and live pitch. The final pitch is five minutes followed by five minutes of questions.

event item	expectation
team	2–4 members; our team of four fits
submission	public repo, clear readme, demo video, and track-specific criteria
demo	live working product, not only slides or a concept
originality	new project built during the official hacking period
judging	product UX, real-world readiness, technical implementation, and presentation

the four judging pillars

pillar	how treasury shield satisfies it
product	a focused treasury-risk product with a clear user, a defined outcome, and Thetanuts options at the center of the workflow
user experience	plain-language inputs, one recommended strategy, visible price-drop scenarios, and a before/after result a non-trader can understand quickly
real-world problem	web3 teams have volatile treasury assets but fixed payroll and operating costs; the app helps them measure and discuss that risk
complete implementation	deployed end-to-end flow: setup, live Thetanuts data, calculations, recommendation, simulation, local saving, error states, and responsive screens
presentation	Alice and Bob make the pain memorable; the live demo shows the risk, the hedge, and the outcome in one coherent five-minute story

suggested five-minute pitch

- 0:00–0:45: tell Alice and Bob’s treasury-runway story.
- 0:45–1:30: show the problem with a 10 ETH treasury and fixed expenses.
- 1:30–3:30: run the Treasury Shield setup, scenarios, recommendation, and simulation.
- 3:30–4:15: explain why Thetanuts data and options are essential.
- 4:15–5:00: explain the market, security choices, and what comes next.

closing pitch

“Treasury Shield is not another options terminal. It is a runway decision tool for web3 teams. We turn a volatile wallet balance into an understandable protection plan, powered by live Thetanuts options data.”

acronym key

term	plain-language meaning
AI	artificial intelligence: software that can interpret information and generate decisions or text
API	application programming interface: a way for software systems to communicate
BTC	bitcoin: a digital asset and the network that transfers it
DAO	decentralized autonomous organization: an online organization coordinated by smart contracts and community rules
ETH	ether: the digital asset used to pay for activity on the Ethereum network
MVP	minimum viable product: the smallest complete version that proves the main idea
RPC	remote procedure call: a connection that lets an app read data from a blockchain node
SDK	software development kit: a set of tools and code for building with a platform
UI / UX	user interface / user experience: what the app looks like and how easy it is to use
USDC	USD Coin: a stablecoin designed to track the value of one US dollar
Web3	a broad term for internet applications that use blockchain-based ownership, identity, or payments

sources and boundaries

Official event source: MUBA Blockchain Hackathon 2026 Opening Ceremony PDF, supplied by the team. Builder source: Thetanuts MUBA Hackathon Builder Docs, supplied by the team. Current protocol method names, addresses, and supported workflows must be checked against the live Thetanuts documentation and repository before implementation.

This is a product concept, not financial advice. The MVP should show simulated outcomes and make no promise of profit or guaranteed protection.